Лабораторна робота №4
Створення застосунку на мові Java для роботи з СУБД

Виконав: Лук’янов Герман Анатолійович
Група: БЗ-121-23-1-ПІ
Дата: 26 січня 2026 р.

Мета роботи

Опанування методів та принципів створення застосунків для забезпечення взаємодії з СКБД для вибірки, впорядкування й обробляння даних.

Завдання:
1. Розглянути засоби розроблення програмних комплексів з використанням можливостей роботи з СКБД
2. Створити GUI застосунок для роботи з базою даних
3. Реалізувати функції:
- Сортування інформації за обраним полем з вибором порядку (ASC/DESC)
- Виведення топ-10 записів після сортування
- Відбір записів, значення полів яких належать певному діапазону (BETWEEN)
- Внутрішнє з'єднання двох таблиць (INNER JOIN)
- Обчислення середнього, максимального і мінімального значень (AVG, MAX, MIN)
- Групування записів за умовою з використанням GROUP BY та WHERE

Примітка: Оригінальне завдання передбачало використання C# та Windows Forms у середовищі MS Visual Studio. У даній роботі використано Java 21 та Swing GUI як функціональний аналог, оскільки принципи взаємодії з СУБД залишаються ідентичними.

Структура бази даних

За основу взято базу даних `decanat_lab3` з минулої лабораторної роботи, яка містить 5 таблиць:

Таблиця	Опис	Кількість записів
Group	Студентські групи	5
Student	Інформація про студентів	12
Subject	Навчальні дисципліни	10
Teacher	Викладачі	5
Exam	Результати іспитів/заліків	25

1. Стислі теоретичні відомості

1.1 Оператор SELECT

Для вибірки даних використовується оператор SELECT:

```sql
*SELECT [ALL | DISTINCT] [TOP n [PERCENT]] СпісокВибірки*
*FROM Ім'яТаблиці*
*WHERE УмоваВідбору*
*[ORDER BY Ім'яПоля [ASC | DESC]]*
```

Компоненти:
- `SELECT *` - вибирає всі поля
- `DISTINCT` - видаляє дублікати
- `TOP n` / `LIMIT n` - обмежує кількість записів
- `WHERE` - умова відбору
- `ORDER BY` - сортування (ASC - за зростанням, DESC - за спаданням)

1.2 Оператор BETWEEN (діапазон значень)

Для відбору записів у певному діапазоні:

```sql
*SELECT * FROM Exam*
*WHERE grade BETWEEN 75 AND 95;*
```

Еквівалентно: `WHERE grade >= 75 AND grade <= 95`

1.3 Внутрішнє з'єднання (INNER JOIN)

Об'єднує дві таблиці за спільним полем:

```sql
*SELECT s.last_name, g.group_name*
*FROM Student s*
*INNER JOIN `Group` g ON s.group_id = g.group_id;*
```

Повертає тільки записи з співпадіннями в обох таблицях.

1.4 Агрегатні функції

Функція	Призначення	Приклад
AVG()	Середнє значення	AVG(grade)
MIN()	Мінімальне значення	MIN(grade)
MAX()	Максимальне значення	MAX(grade)
COUNT()	Кількість записів	COUNT(*)
SUM()	Сума значень	SUM(credit_hours)

Приклад:
```sql
SELECT AVG(grade) AS 'Середня оцінка',
MIN(grade) AS 'Мінімум',
MAX(grade) AS 'Максимум'
FROM Exam;
```

1.5 Групування даних (GROUP BY)

Групування записів за полем з застосуванням агрегатних функцій:

```sql
SELECT s.last_name, AVG(e.grade) AS 'Середня оцінка'
FROM Student s
JOIN Exam e ON s.student_id = e.student_id
WHERE e.grade >= 75
GROUP BY s.student_id, s.last_name
HAVING AVG(e.grade) > 85
ORDER BY AVG(e.grade) DESC;
```

1.6 Підключення до БД через JDBC

Рядок підключення (Connection String):
```java
String url = "jdbc:mysql://localhost:3306/decanat_lab3?
useUnicode=true&characterEncoding=UTF-8&serverTimezone=UTC";
String user = "decanat_user";
String password = "decanat123";

Connection conn = DriverManager.getConnection(url, user, password);
```

Виконання запиту:

```java
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM Student");

while (rs.next()) {
	String lastName = rs.getString("last_name");
	String firstName = rs.getString("first_name");
}
```

2. Хід виконання роботи

2.1 Створення проекту Maven

Крок 1: Генерація структури проекту

```bash
mvn archetype:generate \
	-DgroupId=com.university.decanat \
	-DartifactId=decanat-app \
	-DarchetypeArtifactId=maven-archetype-quickstart \
	-DinteractiveMode=false
```

Крок 2: Налаштування `pom.xml`

```xml
<properties>
	<maven.compiler.source>21</maven.compiler.source>
	<maven.compiler.target>21</maven.compiler.target>
</properties>

<dependencies>
	<!-- MySQL Connector -->
	<dependency>
		<groupId>com.mysql</groupId>
		<artifactId>mysql-connector-j</artifactId>
		<version>8.0.33</version>
	</dependency>
</dependencies>
```

2.2 Створення класу підключення до БД (DatabaseConnection.java)

Реалізовано патерн Singleton для управління підключенням:

```java
public class DatabaseConnection {
	private static Connection connection = null;
	private static final String URL =
		"jdbc:mysql://localhost:3306/decanat_lab3?
		useUnicode=true&characterEncoding=UTF-8&serverTimezone=UTC";
	private static final String USER = "decanat_user";
	private static final String PASSWORD = "decanat123";
	public static Connection getConnection() throws SQLException {
		if (connection == null || connection.isClosed()) {
			try {
				Class.forName("com.mysql.cj.jdbc.Driver");
				connection = DriverManager.getConnection(URL, USER, PASSWORD);
			} catch (ClassNotFoundException e) {
				throw new SQLException("MySQL Driver not found", e);
			}
		}
		return connection;
	}
}
```

2.3 Створення головного вікна застосунку (DecanatApp.java)

Структура GUI:

```
JFrame (800x600)
	└─ JTabbedPane (5 вкладок)
		├─ Tab 1: Сортування
		├─ Tab 2: Фільтр по діапазону
		├─ Tab 3: JOIN таблиць
		├─ Tab 4: Агрегатні функції
		└─ Tab 5: GROUP BY
```

Ініціалізація форми:

```java
public class DecanatApp extends JFrame {
	public DecanatApp() {
		setTitle("Система Деканат - Робота з БД");
		setSize(800, 600);
		setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
		JTabbedPane tabbedPane = new JTabbedPane();
		// Додавання вкладок
		tabbedPane.addTab("📊 Сортування", createSortTab());
		tabbedPane.addTab("🔍 Фільтр по діапазону", createFilterTab());
		tabbedPane.addTab("🔗 JOIN", createJoinTab());
		tabbedPane.addTab("📈 Агрегатні функції", createAggregateTab());
		tabbedPane.addTab("📋 GROUP BY", createGroupByTab());
		add(tabbedPane);
		setVisible(true);
	}
}
```

2.4 Реалізація функції №1: Сортування

Завдання: Реалізувати сортування за обраним полем з вибором порядку + топ-10.

Компоненти вкладки:
- JComboBox для вибору поля (last_name, first_name, grade, enrollment_year, exam_date)
- JComboBox для порядку сортування (ASC, DESC)
- JButton "Сортувати" - всі записи
- JButton "Топ-10" - перші 10 записів
- JTable для відображення результатів

Код реалізації:

```java
private void sortData(String field, String order, boolean top10) {
	try {
		String query = "SELECT * FROM Student ORDER BY " + field + " " + order;
		if (top10) {
			query += " LIMIT 10";
		}
		executeQuery(query);
	} catch (Exception e) {
		JOptionPane.showMessageDialog(this, "Помилка: " + e.getMessage());
	}
}
```

SQL запит (приклад):
```sql
-- Всі записи
SELECT * FROM Student ORDER BY last_name ASC;

-- Топ-10
SELECT * FROM Student ORDER BY last_name ASC LIMIT 10;
```

2.5 Реалізація функції №2: Фільтр по діапазону

Завдання: Відбір записів, значення полів яких належать певному діапазону.

Компоненти:
- JComboBox для вибору поля (grade, enrollment_year, credit_hours)
- JTextField "Від"
- JTextField "До"
- JButton "Фільтрувати"
- JTable для результатів

Код реалізації:

```java
private void filterByRange(String field, String from, String to) {
	try {
		String query = "SELECT s.last_name, s.first_name, e." + field +
				" FROM Student s JOIN Exam e ON s.student_id = e.student_id" +
				" WHERE e." + field + " BETWEEN " + from + " AND " + to +
				" ORDER BY e." + field;
		executeQuery(query);
	} catch (Exception e) {
		JOptionPane.showMessageDialog(this, "Помилка: " + e.getMessage());
	}
}
```

SQL запит (приклад):
```sql
SELECT s.last_name, s.first_name, e.grade
FROM Student s
JOIN Exam e ON s.student_id = e.student_id
WHERE e.grade BETWEEN 75 AND 95
ORDER BY e.grade DESC;
```

2.6 Реалізація функції №3: Внутрішнє з'єднання (INNER JOIN)

Завдання: Реалізувати внутрішнє з'єднання двох таблиць.

Компоненти:
- JButton "Студенти + Групи" (2 таблиці)
- JButton "Повний JOIN" (5 таблиць)
- JButton "Студенти + Іспити" (з агрегацією)
- JTable для результатів

Код реалізації:

```java
private void joinStudentGroup() {
	String query = """
		SELECT s.student_id, s.last_name, s.first_name,
			g.group_name, g.course, g.specialty
		FROM Student s
		INNER JOIN `Group` g ON s.group_id = g.group_id
		ORDER BY g.group_name, s.last_name
		""";
	executeQuery(query);
}

private void joinAllTables() {
	String query = """
		SELECT
			CONCAT(s.last_name, ' ', LEFT(s.first_name, 1), '.') AS 'Студент',
			g.group_name AS 'Група',
			sub.subject_name AS 'Предмет',
			CONCAT(t.last_name, ' ', LEFT(t.first_name, 1), '.') AS 'Викладач',
			e.grade AS 'Оцінка',
			e.exam_date AS 'Дата'
		FROM Exam e
		INNER JOIN Student s ON e.student_id = s.student_id
		INNER JOIN `Group` g ON s.group_id = g.group_id
		INNER JOIN Subject sub ON e.subject_id = sub.subject_id
		INNER JOIN Teacher t ON e.teacher_id = t.teacher_id
		ORDER BY e.exam_date DESC
		LIMIT 20
		""";
	executeQuery(query);
}
```

SQL запит (приклад):
```sql
-- JOIN 2 таблиць
SELECT s.last_name, s.first_name, g.group_name
FROM Student s
INNER JOIN `Group` g ON s.group_id = g.group_id;

-- JOIN 5 таблиць
SELECT s.last_name, g.group_name, sub.subject_name,
t.last_name AS teacher, e.grade
FROM Exam e
INNER JOIN Student s ON e.student_id = s.student_id
INNER JOIN `Group` g ON s.group_id = g.group_id
INNER JOIN Subject sub ON e.subject_id = sub.subject_id
INNER JOIN Teacher t ON e.teacher_id = t.teacher_id;
```

2.7 Реалізація функції №4: Агрегатні функції (AVG, MIN, MAX)

Завдання: Визначення середнього, максимального і мінімального значень для числових полів.

Компоненти:
- JComboBox вибору таблиці (Exam, Subject, Student)
- JButton "Середнє (AVG)"
- JButton "Мінімум (MIN)"
- JButton "Максимум (MAX)"
- JButton "Всі статистики"
- JTextArea для результатів

Код реалізації:

```java
private void calculateAggregate(String function, int tableIndex) {
	try {
		Connection conn = DatabaseConnection.getConnection();
		Statement stmt = conn.createStatement();
		String query = "";
		switch (tableIndex) {
			case 0: // Exam
				query = "SELECT " + function + "(grade) FROM Exam";
				break;
			case 1: // Subject
				query = "SELECT " + function + "(credit_hours) FROM Subject";
				break;
			case 2: // Student
				query = "SELECT " + function + "(enrollment_year) FROM Student";
				break;
```

```
            }
            ResultSet rs = stmt.executeQuery(query);
            if (rs.next()) {
                double result = rs.getDouble(1);
                resultArea.append(function + ": " + result + "\n");
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Помилка: " + e.getMessage());
        }
}

private void showAllStatistics() {
        try {
            Connection conn = DatabaseConnection.getConnection();
            Statement stmt = conn.createStatement();
            // Статистика по оцінках
            ResultSet rs = stmt.executeQuery("""
                SELECT COUNT(*) as total, AVG(grade) as avg,
                    MIN(grade) as min, MAX(grade) as max
                FROM Exam
                """);
            if (rs.next()) {
                resultArea.append("=== СТАТИСТИКА ПО ОЦІНКАХ ===\n");
                resultArea.append("Всього іспитів: " + rs.getInt("total") + "\n");
                resultArea.append("Середня оцінка: " + String.format("%.2f",
                rs.getDouble("avg")) + "\n");
                resultArea.append("Мінімальна оцінка: " + rs.getInt("min") + "\n");
                resultArea.append("Максимальна оцінка: " + rs.getInt("max") + "\n\n");
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Помилка: " + e.getMessage());
        }
}
```

SQL запити (приклад):

```sql
-- Середнє значення
SELECT AVG(grade) AS 'Середня оцінка' FROM Exam;

-- Мінімум
SELECT MIN(grade) AS 'Мінімальна оцінка' FROM Exam;

-- Максимум
SELECT MAX(grade) AS 'Максимальна оцінка' FROM Exam;
```

```
-- Всі разом
SELECT
	COUNT(*) AS 'Всього',
	AVG(grade) AS 'Середня',
	MIN(grade) AS 'Мінімум',
	MAX(grade) AS 'Максимум'
	FROM Exam;
```

2.8 Реалізація функції №5: GROUP BY з WHERE

Завдання: Групування записів за умовою з використанням GROUP BY та WHERE.

Компоненти:
- JTextField для введення мінімальної оцінки (WHERE умова)
- JButton "Середні оцінки студентів"
- JButton "Середні оцінки по предметах"
- JButton "Середні оцінки по групах"
- JTable для результатів

Код реалізації:

```java
private void groupByStudent(String minGrade) {
	String query = """
		SELECT
			s.last_name AS 'Прізвище',
			s.first_name AS 'Ім\'я',
			COUNT(e.exam_id) AS 'Іспитів',
			AVG(e.grade) AS 'Середня оцінка'
		FROM Student s
		JOIN Exam e ON s.student_id = e.student_id
		WHERE e.grade >= """ + minGrade + """
		GROUP BY s.student_id, s.last_name, s.first_name
		HAVING COUNT(e.exam_id) > 0
		ORDER BY AVG(e.grade) DESC
		""";
	executeQuery(query);
}

private void groupBySubject(String minGrade) {
	String query = """
		SELECT
			sub.subject_name AS 'Дисципліна',
			sub.credit_hours AS 'Кредити',
			COUNT(e.exam_id) AS 'Іспитів',
			AVG(e.grade) AS 'Середня оцінка'
```

```
                FROM Subject sub
                JOIN Exam e ON sub.subject_id = e.subject_id
                WHERE e.grade >= """ + minGrade + """
                GROUP BY sub.subject_id, sub.subject_name, sub.credit_hours
                HAVING COUNT(e.exam_id) > 0
                ORDER BY AVG(e.grade) DESC
                """;
        executeQuery(query);
}

private void groupByGroup(String minGrade) {
        String query = """
                SELECT
                        g.group_name AS 'Група',
                        g.course AS 'Курс',
                        COUNT(DISTINCT s.student_id) AS 'Студентів',
                        COUNT(e.exam_id) AS 'Іспитів',
                        AVG(e.grade) AS 'Середня оцінка'
                FROM `Group` g
                JOIN Student s ON g.group_id = s.group_id
                JOIN Exam e ON s.student_id = e.student_id
                WHERE e.grade >= """ + minGrade + """
                GROUP BY g.group_id, g.group_name, g.course
                HAVING COUNT(e.exam_id) > 0
                ORDER BY AVG(e.grade) DESC
                """;
        executeQuery(query);
}
```

SQL запити (приклад):

```sql
-- Групування по студентах з WHERE
SELECT
        s.last_name,
        COUNT(e.exam_id) AS 'Іспитів',
        AVG(e.grade) AS 'Середня оцінка'
FROM Student s
JOIN Exam e ON s.student_id = e.student_id
WHERE e.grade >= 75
GROUP BY s.student_id, s.last_name
HAVING AVG(e.grade) > 85
ORDER BY AVG(e.grade) DESC;
```

```
-- Групування по предметах
SELECT
        sub.subject_name,
        COUNT(e.exam_id) AS 'Іспитів',
        AVG(e.grade) AS 'Середня оцінка'
FROM Subject sub
JOIN Exam e ON sub.subject_id = e.subject_id
WHERE e.grade >= 75
GROUP BY sub.subject_id, sub.subject_name;

-- Групування по групах
SELECT
        g.group_name,
        COUNT(DISTINCT s.student_id) AS 'Студентів',
        AVG(e.grade) AS 'Середня оцінка'
FROM `Group` g
JOIN Student s ON g.group_id = s.group_id
JOIN Exam e ON s.student_id = e.student_id
WHERE e.grade >= 75
GROUP BY g.group_id, g.group_name;
```

2.9 Компіляція та запуск проекту

****Компіляція:****
```bash
mvn clean compile
```

Створення JAR з залежностями:
```bash
mvn clean package
```

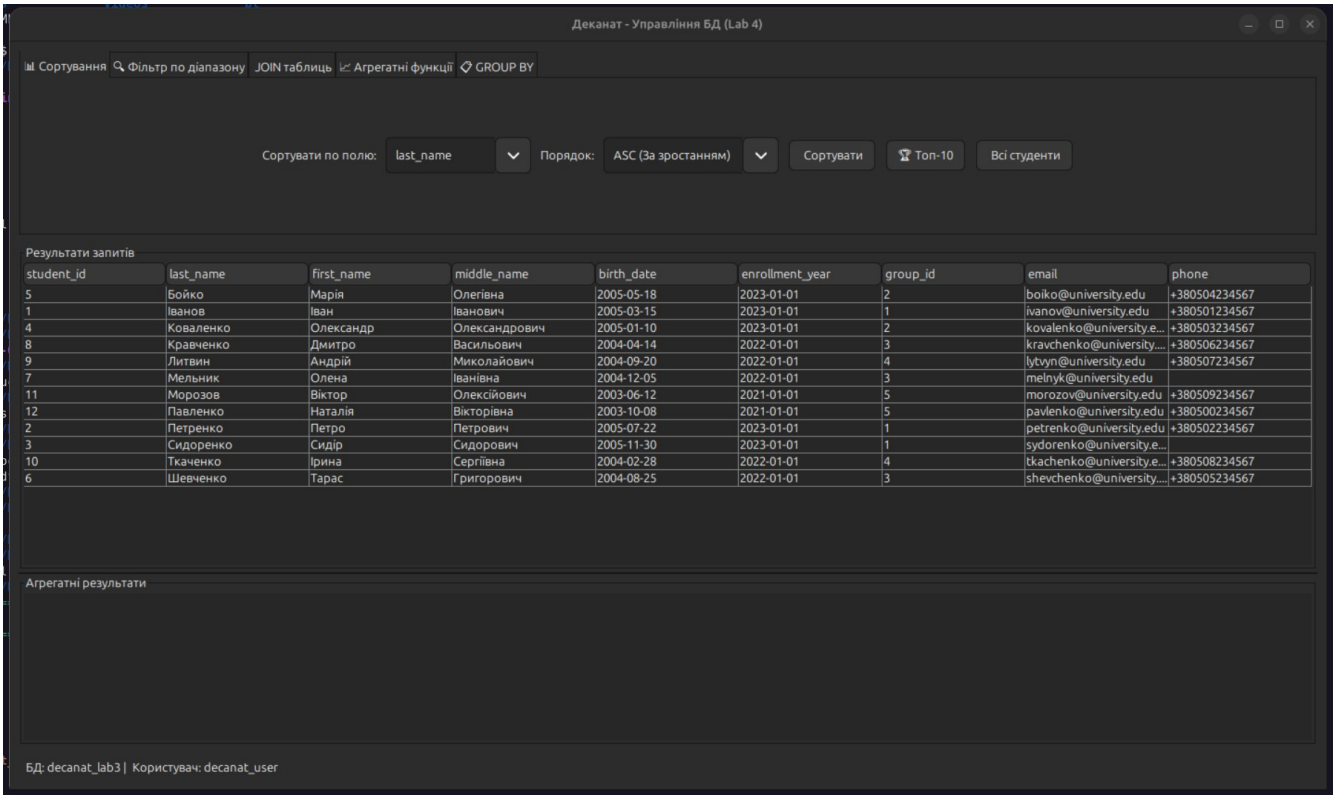
Запуск застосунку:
```bash
# Через Maven
mvn exec:java -Dexec.mainClass="com.university.decanat.DecanatApp"

# Або через JAR файл
java -jar target/decanat-app.jar
```

3. Демонстрація роботи програми

3.1 Головне вікно застосунку і вкладка “Сортування”

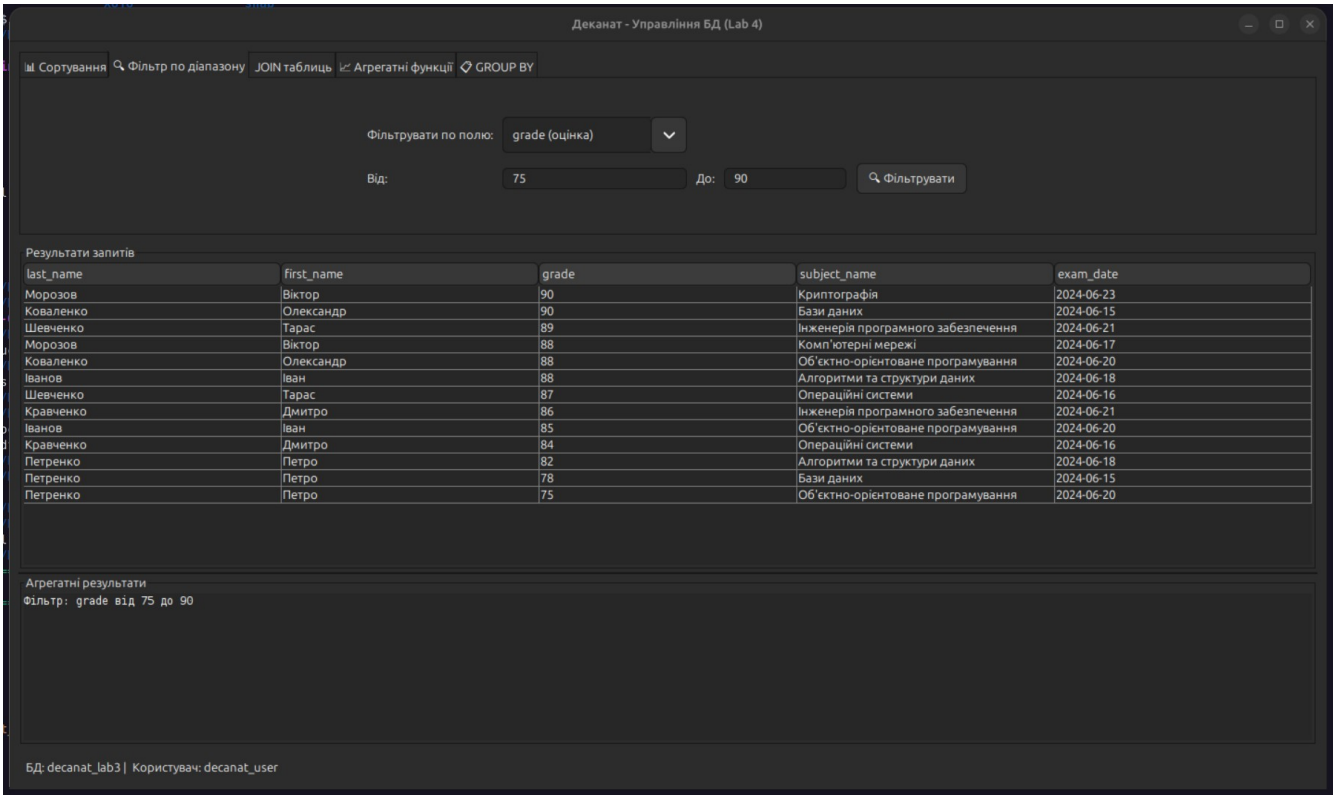
Скріншот 1: Головне вікно з вкладками, Інтерфейс сортування



Опис:
Головне вікно з 5 вкладками для різних функцій роботи з БД.
Користувач обирає поле сортування (last_name), порядок (ASC), натискає кнопку "Топ-10".

3.2 Вкладка "Фільтр по діапазону"

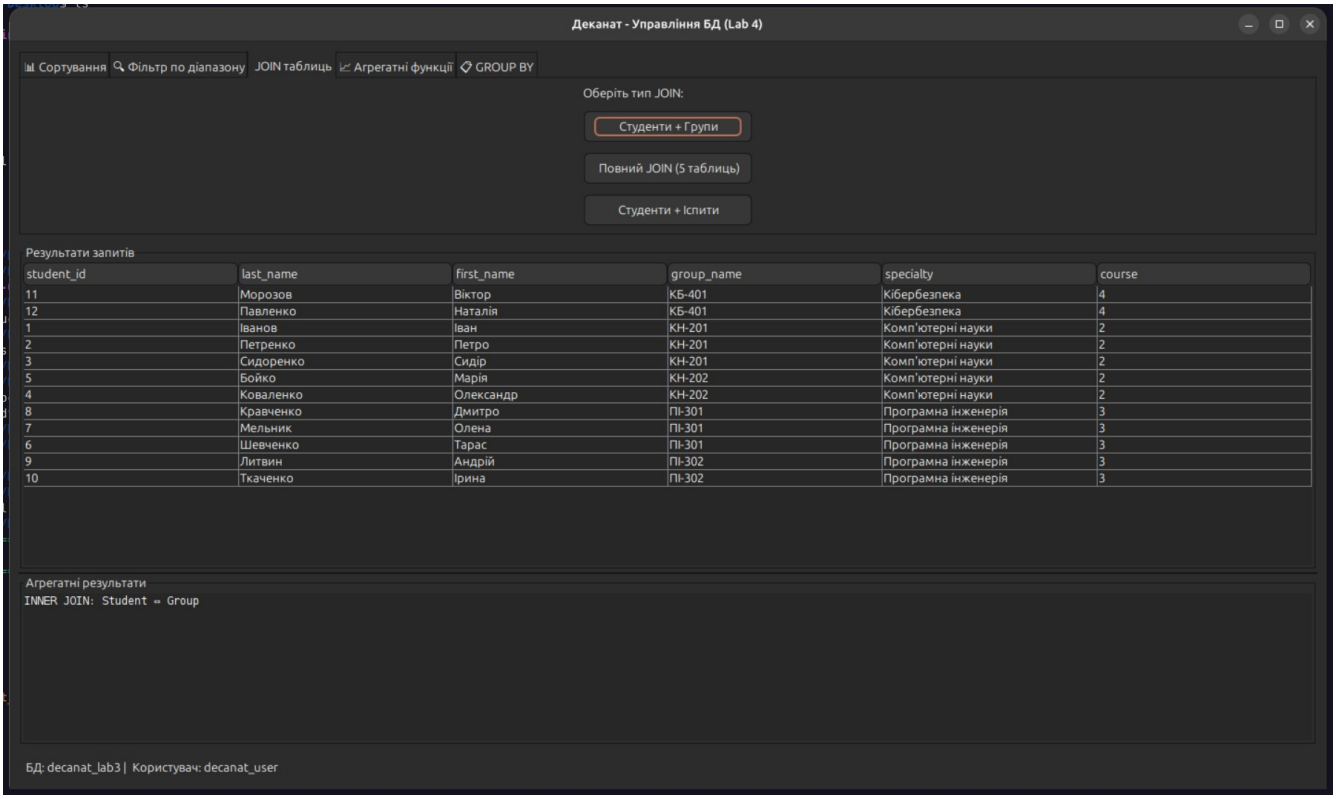
Скріншот 2: Інтерфейс фільтрації



Опис: Користувач вводить діапазон оцінок від 75 до 95, натискає "Фільтрувати".

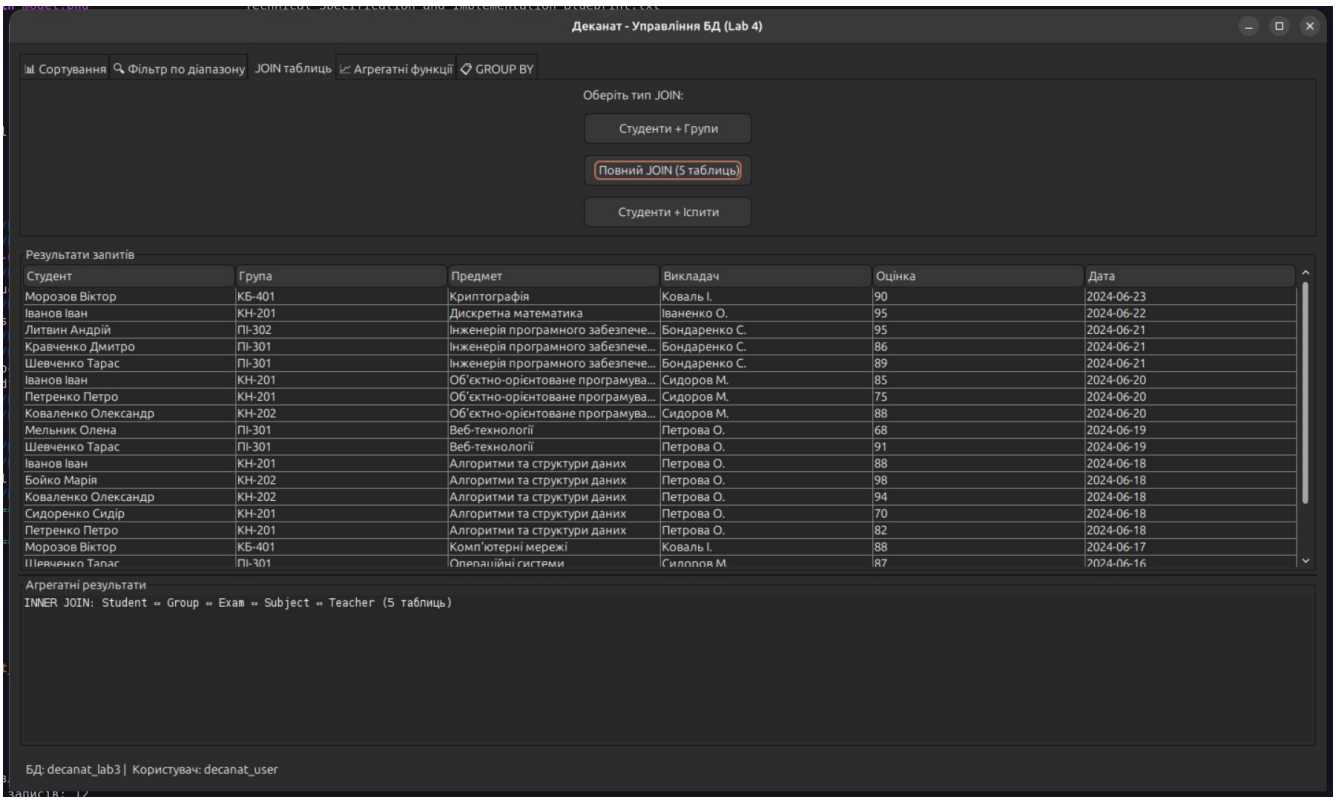
3.3 Вкладка "JOIN таблиць"

Скріншот 3: Результат INNER JOIN (2 таблиці)



Опис: Натиснута кнопка "Студенти + Групи" - показує студентів з назвами груп.

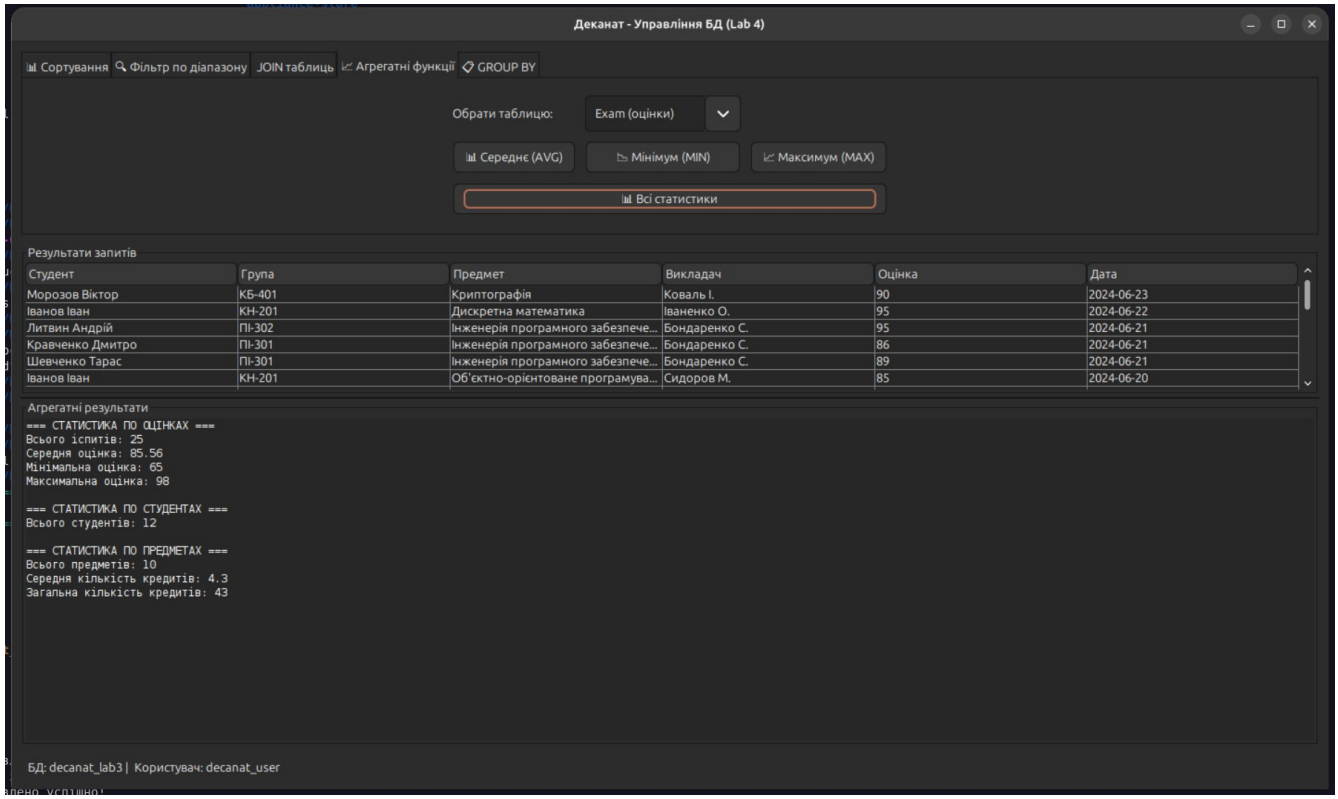
Скріншот 4: Результат JOIN 5 таблиць



Опис: Натиснута кнопка "Повний JOIN" - показує повну інформацію про іспити з усіх 5 таблиць.

3.4 Вкладка "Агрегатні функції"

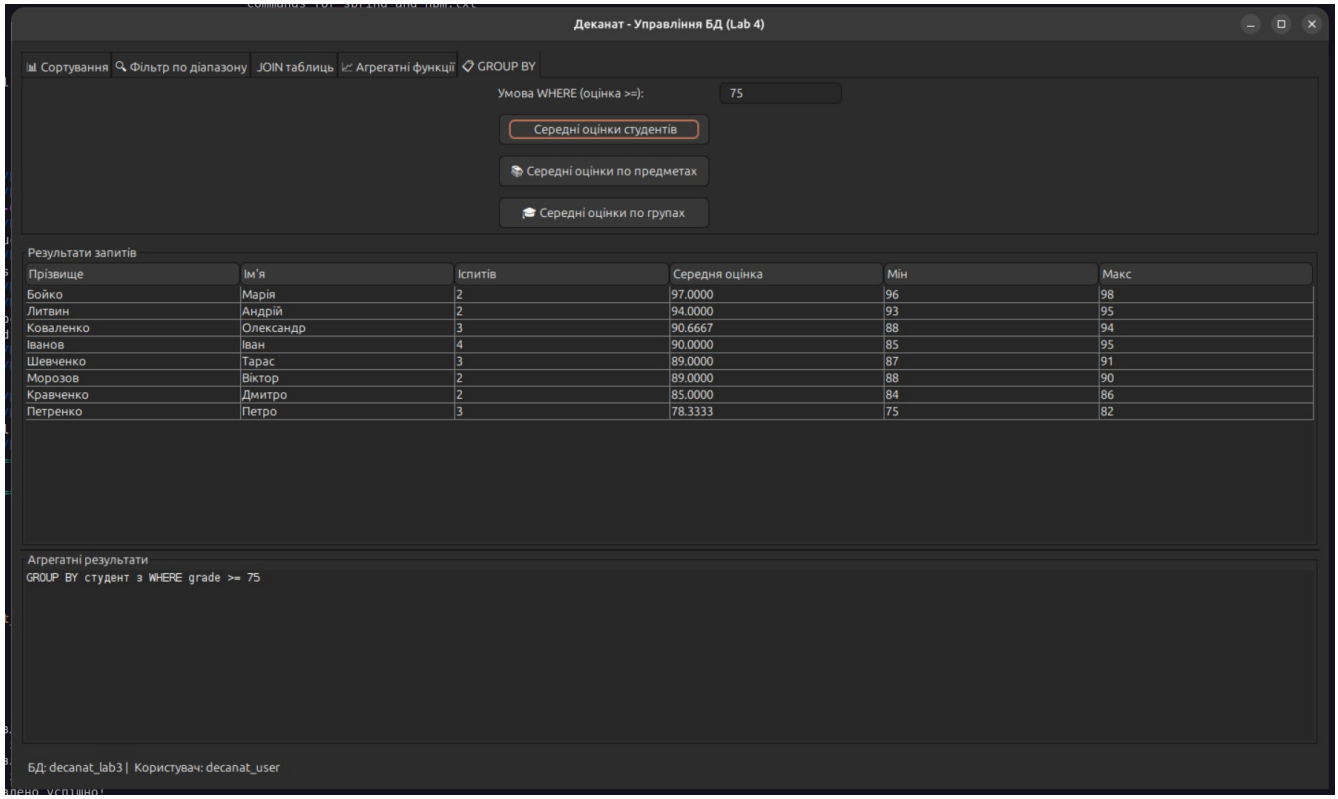
Скріншот 5: Обчислення AVG, MIN, MAX



Опис: Натиснута кнопка "Всі статистики" - показує комплексну статистику.

3.5 Вкладка "GROUP BY"

Скріншот 6: Групування з WHERE умовою



Опис: Введено мінімальну оцінку 75, натиснута кнопка "Середні оцінки студентів".

Висновки

Досягнуті результати

У ходів виконання лабораторної роботи №4 було успішно:

1. **Створено повнофункціональний застосунок** на мові Java для роботи з СУБД MySQL, який реалізує всі вимоги завдання

2. **Освоєно основні методи взаємодії з СУБД:**
- Підключення до БД через JDBC (MySQL Connector/J 8.0.33)
- Виконання SQL запитів через Statement та ResultSet
- Обробка результатів запитів та відображення їх у GUI

3. **Реалізовано 5 основних функцій:**

А) Сортування інформації:
- Вибір поля для сортування (last_name, first_name, grade, enrollment_year, exam_date)
- Вибір порядку сортування (ASC - за зростанням, DESC - за спаданням)
- Відображення всіх записів або топ-10
- Використано оператор `ORDER BY` та `LIMIT`

Б) Фільтрація по діапазону:
- Відбір записів за значенням полів у заданому діапазоні
- Реалізовано для полів: grade (оцінка), enrollment_year (рік вступу), credit_hours (кредити)
- Використано оператор `BETWEEN`
- Автоматичне формування SQL запиту на основі вводу користувача

В) Внутрішнє з'єднання таблиць (INNER JOIN):
- JOIN 2 таблиць: Student + Group
- JOIN 5 таблиць: Exam + Student + Group + Subject + Teacher
- Відображення повної інформації про іспити з усіх пов'язаних таблиць

Г) Агрегатні функції:
- Обчислення середнього значення (AVG)
- Знаходження мінімуму (MIN)
- Знаходження максимуму (MAX)
- Підрахунок кількості записів (COUNT)
- Обчислення суми (SUM)
- Комплексна статистика по всіх таблицях

Д) Групування з умовою (GROUP BY + WHERE):
- Групування по студентах з середніми оцінками
- Групування по дисциплінах зі статистикою іспитів
- Групування по групах з загальною успішністю
- Використання WHERE для фільтрації записів перед групуванням
- Використання HAVING для фільтрації груп після агрегації

4. **Створено зручний графічний інтерфейс:**
- Використано Java Swing (аналог Windows Forms)
- Вкладковий інтерфейс (JTabbedPane) для організації функцій
- Таблиці для відображення результатів (JTable)

- Елементи вводу даних (JTextField, JComboBox)
- Кнопки для виконання операцій (JButton)
- Панель статусу для відображення повідомлень